Rapport — Éditeur collaboratif

Auteurs

- NICAISE Enzo
- NGUYEN Ngoc Dang Nguyen

Organisation du code

Tout le code serveur et les clients automatiques sont dans `editor/src/main/java/`, compilés directement avec `javac`. Les fichiers suivent la progression du projet :

Fichier	Rôle
`ServerCentral.java`	Serveur centralisé, mode pull
`ServerCentralPush.java`	Serveur centralisé, notifications push
`ServerFederated.java`	Serveur fédéré pair-à-pair
`ServerMaster.java`	Fédération avec maître-esclaves
`ServerMasterFaulty.java`	Maître instable (démonstration SPOF)
`ServerDispatch.java`	Répartiteur de charge (round-robin)
`ServerRaft.java`	Cluster tolérant aux pannes (Raft, 3 nœuds)
`AutoClient.java`	Client automatique pour les tests de convergence
`BenchClient.java`	Client de benchmark (latence et débit)
`GUIClient.java`	Client graphique JavaFX
`ClientController.java`	Client graphique JavaFX

Protocole. Toutes les communications client-serveur et serveur-serveur utilisent un protocole texte sur TCP (une commande par ligne). Les commandes principales sont `GETD`, `ADDL`, `MDFL`, `RMVL` côté client, et `LINE`, `DONE`, `ERRL` côté serveur. Ce choix simplifie le débogage (netcat suffit pour tester un serveur à la main) et l'écriture des clients automatiques.

Modèle de threads. Chaque connexion entrante est gérée par un thread dédié (`Thread per client`). Pour les serveurs fédérés et Raft, les connexions inter-nœuds suivent le même modèle. La synchronisation sur le document partagé est assurée par `synchronized` sur l'objet serveur (tâches 1–6) ou par le thread `applyLoop` unique dans Raft.

Le client graphique est géré séparément via Gradle (`editor/build.gradle`) car JavaFX nécessite des dépendances Maven. L'interface est définie dans `editor/src/main/resources/clientView.fxml`.

Les scripts de validation et de benchmark sont dans `editor/scripts/`. Chaque script est autonome : il compile, démarre les serveurs sur des ports locaux, exécute les clients avec barrière de synchronisation, vérifie la convergence, puis affiche le résultat.

Tâche 1 — Serveur centralisé

La première question à régler était le choix de la structure de données et la gestion des accès concurrents entre clients. Le document partagé est représenté par une `List<String>`, une entrée par ligne. On y associe une `List<Integer>` de numéros de version ligne par ligne, ce qui permet de détecter les modifications concurrentes côté client. Les deux listes sont protégées par un bloc `synchronized` sur l'instance serveur pour éviter les accès simultanés.

En mode pull, c'est le client qui envoie `GETD` pour récupérer les modifications des autres. Le client applique ses propres changements localement dès l'envoi, sans attendre la réponse du serveur. Ça marche pour l'utilisateur local, mais les autres clients ne voient rien avant leur prochain rafraîchissement.

Les conflits sont réglés par ordre d'arrivée au serveur : si deux clients modifient la même ligne en même temps, c'est le premier arrivé qui gagne. Ce n'est pas idéal, mais c'est simple et ça suffit pour les besoins du projet.

Tâche 2 — Serveur push

En mode pull, les modifications des autres n'apparaissent qu'après un rafraîchissement manuel, ce qui n'est pas très pratique. Le `ServerCentralPush` résout ça en notifiant directement tous les clients connectés à chaque modification — uniquement la ligne concernée, pas tout le document.

Côté client, un thread de lecture tourne en permanence. Il fait la différence entre les réponses à un `GETD` (qu'il accumule jusqu'au `DONE`) et les notifications push qui arrivent spontanément. Pour la validation, on a écrit `AutoClient` : chaque instance fait un certain nombre d'opérations aléatoires, puis attend que tous les autres aient fini grâce à une barrière à base de fichiers, avant de faire un `GETD` final. Avec 8 clients faisant 20 opérations chacun, tous les documents finaux sont identiques.

Tâche 3 — Interconnexion de serveurs

La première question était de savoir si un serveur pouvait simplement se connecter à un autre comme un client normal. `GETD` fonctionne pour récupérer l'état initial, mais ensuite il n'y a aucun moyen d'éviter les boucles : A notifie B, qui renvoie à A, etc.

On a résolu ça avec un message `PEERSYNC` envoyé juste après la connexion, qui marque la connexion entrante comme étant un pair et non un client ordinaire. Les modifications sont propagées aux pairs sous la forme `SYNC ADDL ...` plutôt que `ADDL ...` directement, et un serveur qui reçoit un `SYNC` l'applique sans le re-propager. La connexion sortante est déclenchée par la commande `CONNECT host port` envoyée par un script au démarrage.

Sur deux nœuds avec un client par serveur et une barrière à 2, les deux documents finaux sont identiques. La limite apparaît quand deux clients modifient la même ligne sur deux serveurs différents en même temps : l'ordre de réception des `SYNC` est imprévisible, et les deux serveurs peuvent finir avec des contenus différents.

Tâche 4 — Fédération simple

Avec l'interconnexion fonctionnelle, on a pu passer à des topologies plus complètes. On a testé deux configurations : deux serveurs (B → A) et trois serveurs en étoile (B → A ← C). Dans la topologie étoile, A reçoit les `SYNC` de B et les retransmet à C, et vice-versa, en excluant la source pour ne pas créer de boucle.

Avec un client par serveur et une barrière à 3, les documents convergent dans tous nos tests. Mais ça reste fragile : la convergence tient uniquement parce que nos clients n'écrivent pas sur la même ligne en même temps. Dès qu'on a forcé deux clients à modifier la même ligne simultanément sur deux serveurs différents, les documents ont divergé. C'est ce qui nous a poussé à ajouter un serveur maître.

Tâche 5 — Fédération avec serveur maître

L'idée est simple : un seul nœud décide de l'ordre de toutes les modifications, les autres exécutent. La configuration se fait dans `peers.cfg`, qui ne contient qu'une seule ligne significative : `master = host port`. Chaque serveur lit ce fichier au démarrage et détermine lui-même son rôle : si son propre port d'écoute correspond au port du maître, il joue le rôle de maître ; sinon, il initie automatiquement une connexion vers le maître et se comporte en esclave.

Quand un client envoie une modification à un esclave, l'esclave la transfère au maître avec `FWDL reqId cmd` et attend. Le maître l'applique, lui assigne un numéro de séquence, et diffuse `ORDER seqno reqId cmd` à tous les esclaves. Chaque esclave applique la modification dans cet ordre, notifie ses clients en push, et débloque le client en attente via le `reqId`. Tous les serveurs voient donc exactement les mêmes modifications dans le même ordre.

Sur nos tests (1 maître + 2 esclaves, 3 clients simultanés), les trois documents finaux sont toujours identiques, même quand plusieurs clients modifient la même ligne.

Tâche 6 — Validation et montée en charge

Pour que les clients n'aient pas à connaître l'adresse d'un serveur précis, on a ajouté `ServerDispatch`. Il écoute sur un port dédié et répond à `GETSERVER` par l'adresse d'un serveur choisi en round-robin depuis `dispatch.cfg`. Le client se connecte ensuite directement à ce serveur. La validation fonctionnelle avec 6 clients `BenchClient` connectés via le dispatch à une fédération 1 maître + 2 esclaves donne des documents identiques dans tous les cas.

Pour les benchmarks, `BenchClient` envoie des opérations sans pause (`thinkMs = 0`). Le script `bench.sh` produit `bench_results.csv`, tracé par `plot-bench.py`. Les mesures sont réalisées sur une seule machine en local (processus Java sur loopback TCP) ; les valeurs absolues de débit sont indicatives et peuvent varier selon la charge système et le warmup JVM — c'est la tendance qui importe.

Scénario A — nombre de clients variable, 3 serveurs fixes (1 maître + 2 esclaves)

Clients	Latence moy. (ms)	Débit total (ops/s)
1	3,4	167
2	4,0	196
4	3,7	686
8	4,0	1025

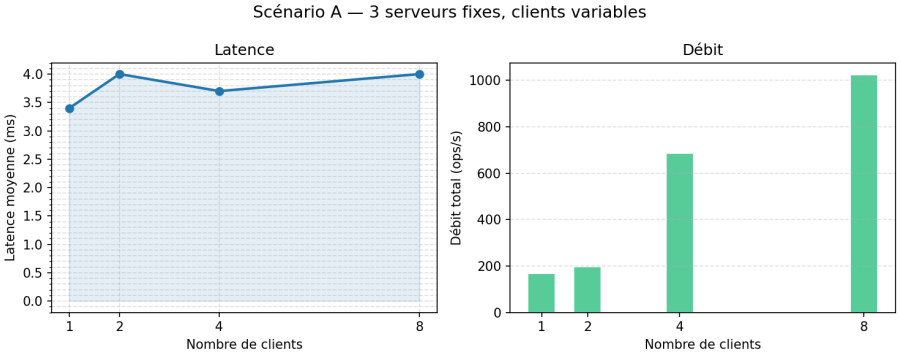

Figure 1: Scénario A — latence et débit en fonction du nombre de clients

La latence reste stable autour de 3,5–4 ms quel que soit le nombre de clients. Le chemin d'une opération est toujours le même (client → esclave → maître → ORDER → réponse), et sa durée est surtout dictée par les allers-retours TCP locaux. Le débit augmente avec le nombre de clients, ce qui s'explique par le traitement parallèle des requêtes. La légère anomalie à 2 clients (débit proche de 1 client) vient du fait que le dispatch en assigne un au maître et un à un esclave, et ce deuxième subit le cycle `FWDL → ORDER` supplémentaire qui accentue les variations entre clients.

Scénario B — nombre de serveurs variable, 4 clients fixes

Serveurs	Latence moy. (ms)	Débit total (ops/s)
2	3,5	442
3	3,3	686

Serveurs	Latence moy. (ms)	Débit total (ops/s)
5	4,3	633
10	7,2	408

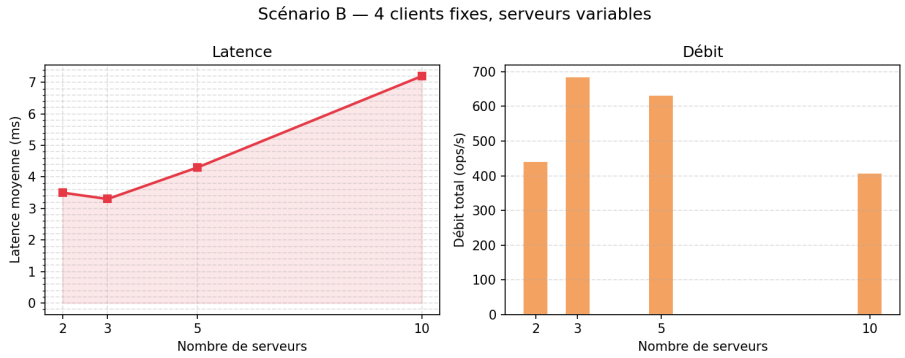

Figure 2: Scénario B — latence et débit en fonction du nombre de serveurs

La tendance la plus claire est sur la latence : elle passe de 3,3 ms à 3 serveurs à 7,2 ms à 10 serveurs. Avec 4 clients répartis en round-robin sur 10 serveurs, la plupart atterrissent sur des esclaves et subissent le cycle `FWDL → maître → ORDER` supplémentaire — le maître traite tous ces forwards en série et devient le goulot d'étranglement. Le débit ne suit pas une tendance monotone : il est maximal à 3 serveurs (686 ops/s) car le dispatch répartit équitablement les 4 clients, puis redescend quand le nombre d'esclaves augmente et que le maître est plus sollicité. Ajouter des esclaves aide pour la distribution des connexions, mais pas pour les écritures.

Tâche 7 — Défaillance

Tâche 7A — Point de défaillance unique

Pour illustrer le problème, on a créé `ServerMasterFaulty`. Toutes les 15 secondes, le maître se fige pendant 5 secondes (il ignore toutes les requêtes sans répondre), puis se coupe définitivement après 3 minutes.

Ce qu'on observe : pendant les phases de gel, les clients connectés à des esclaves envoient `FWDL` et n'obtiennent jamais de réponse — ils restent bloqués. Quand le maître s'arrête définitivement, les esclaves perdent la connexion et refusent toute nouvelle écriture. Le cluster s'arrête alors même que les esclaves sont tous encore actifs. C'est exactement le comportement d'un système avec un point de défaillance unique.

Tâche 7B — Algorithme Raft

Pour éviter ce problème, on a implémenté Raft dans `ServerRaft` pour 3 nœuds. L'idée est qu'aucun nœud n'a de rôle fixe : n'importe lequel peut devenir leader si le leader actuel disparaît.

Chaque nœud utilise deux ports : un port client (5000/5001/5002) pour les `AutoClient`, et un port Raft interne (`clientPort + 1000`) pour les communications entre nœuds. Les nœuds se connectent entre eux au démarrage.

Élection. Chaque nœud tire un timeout aléatoire entre 1500 et 3000 ms. S'il n'entend pas de heartbeat dans ce délai, il se déclare candidat, incrémente son terme et envoie `RequestVote` aux autres. Avec une majorité de votes (2 sur 3), il devient leader et envoie des heartbeats toutes les 300 ms. Un nœud ne vote que pour un candidat dont le log est au moins aussi à jour que le sien.

Réplication. Quand le leader reçoit une écriture, il l'ajoute à son log et l'envoie aux followers via `AppendEntries`. Une fois qu'une majorité a confirmé la réception, l'entrée est committée et appliquée au document, puis diffusée aux clients. Les followers committent à leur tour lors du prochain `AppendEntries`.

On a eu un bug assez pénible à trouver : le format initial de `AppendEntries` collait la première entrée directement aux métadonnées sans séparateur, ce qui cassait le parsing lors du `split`. La correction a été d'ajouter un espace explicite avant la première entrée.

Résultats. Avec 3 nœuds et 3 clients faisant 5 opérations chacun, tous les documents finaux sont identiques. Quand on force la panne d'un nœud après 4 secondes, le cluster continue sans interruption — le quorum 2/3 est maintenu. Les 2 clients restants obtiennent le même document, ce qui confirme que la panne d'un nœud sur trois ne pose aucun problème.